

MODUL PRAKTIKUM STRUKTUR DATA

Modul 3



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

MODUL 3

KONSEP STRUKTUR DATA, ADT, DAN ANALISIS KOMPLEKSITAS ALGORITMA

1. DESKRIPSI MODUL

Modul praktikum ini membahas konsep dasar struktur data sebagai fondasi utama dalam pengembangan algoritma dan pemrograman yang efisien. Mahasiswa mempelajari cara data disimpan, diorganisasikan, dikelola, dan diproses agar pengolahan data dapat dilakukan secara optimal.

Selain itu, modul ini memperkenalkan Abstract Data Type (ADT) sebagai representasi konseptual suatu struktur data, serta analisis efisiensi algoritma menggunakan Big-O Notation untuk membandingkan performa program. Pemahaman materi ini menjadi dasar bagi implementasi struktur data linear maupun non-linear pada pertemuan berikutnya.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Memahami konsep dasar struktur data.
- Menjelaskan pengertian Abstract Data Type (ADT).
- Memahami hubungan struktur data, algoritma, dan efisiensi program.
- Mengenali dan menganalisis kompleksitas waktu algoritma sederhana.
- Menggunakan Big-O Notation untuk menggambarkan pertumbuhan waktu eksekusi.
- Membandingkan efisiensi beberapa algoritma dasar.
- Mengimplementasikan simulasi analisis algoritma menggunakan Python.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

- Mahasiswa mampu menjelaskan konsep struktur data dan ADT.
- Mahasiswa mampu mengidentifikasi efisiensi suatu algoritma.
- Mahasiswa mampu menganalisis kompleksitas waktu sederhana.
- Mahasiswa mampu membandingkan performa algoritma pencarian sederhana.
- Mahasiswa mampu mengembangkan pola berpikir logis dan analitis melalui implementasi Python.

4. DASAR TEORI

4.1 Struktur Data

Struktur data adalah cara untuk menyimpan, mengorganisasikan, dan mengelola data di dalam komputer sehingga data dapat digunakan secara efisien. Pemilihan struktur data yang tepat mempengaruhi kecepatan pemrosesan data, efisiensi penggunaan memori, dan kemudahan pengembangan program.

Contoh struktur data:

- Array
- List
- Stack
- Queue
- Tree
- Graph

```
data = [10, 20, 30, 40]
print(data)
```

4.2 Jenis Struktur Data

Secara umum, struktur data dapat dibagi menjadi dua kelompok:

1. Struktur Data Linear

Data disusun secara berurutan. Contoh: Array, List, Stack, dan Queue.

2. Struktur Data Non-Linear

Data tidak selalu disusun secara berurutan, melainkan dapat berbentuk hierarki atau jaringan.

Contoh: Tree dan Graph.

4.3 Abstract Data Type (ADT)

Abstract Data Type (ADT) adalah model konseptual yang mendefinisikan jenis data, operasi yang dapat dilakukan, dan perilaku struktur data tanpa berfokus pada detail implementasi.

Contoh ADT Stack memiliki operasi utama push, pop, dan peek. Contoh ADT Queue memiliki operasi enqueue dan dequeue.

```
stack = []
stack.append(10)    # push
stack.append(20)    # push
print(stack.pop()) # pop
```

4.4 Pentingnya Efisiensi Algoritma

Efisiensi algoritma menunjukkan seberapa cepat dan seberapa hemat memori suatu program dijalankan. Algoritma yang baik cepat diproses, menggunakan memori secara optimal, dan mampu menangani data besar.

4.5 Kompleksitas Algoritma

Kompleksitas algoritma digunakan untuk mengukur efisiensi algoritma. Dua jenis kompleksitas yang umum dianalisis adalah:

- Time Complexity: mengukur pertumbuhan waktu eksekusi terhadap jumlah data.
- Space Complexity: mengukur kebutuhan memori terhadap jumlah data.

4.6 Big-O Notation

Big-O Notation digunakan untuk menggambarkan pertumbuhan waktu eksekusi algoritma terhadap jumlah data. Tabel berikut merangkum kompleksitas umum:

Kompleksitas	Nama	Contoh
$O(1)$	Konstan	Akses elemen array/list
$O(\log n)$	Logaritmik	Binary Search
$O(n)$	Linear	Linear Search / traversal
$O(n^2)$	Kuadratik	Nested loop / Bubble Sort

```
data = [1, 2, 3, 4, 5]
for x in data:
    print(x)
```

Perulangan di atas berjalan sebanyak jumlah data sehingga kompleksitasnya $O(n)$.

```
for i in range(5):
    for j in range(5):
        print(i, j)
```

Karena terdapat perulangan di dalam perulangan, jumlah proses meningkat secara kuadratik sehingga kompleksitasnya $O(n^2)$.

4.7 Linear Search

Linear search melakukan pencarian dengan memeriksa data satu per satu sampai data ditemukan atau seluruh data selesai diperiksa. Kompleksitas waktu linear search adalah $O(n)$.

```
data = [10, 20, 30, 40]
target = 30

for i in data:
    if i == target:
        print("Data ditemukan")
```

4.8 Binary Search

Binary search bekerja dengan membagi data menjadi dua bagian secara berulang. Syarat utama binary search adalah data harus dalam keadaan terurut. Kompleksitas waktu binary search adalah $O(\log n)$.

```
data = [10, 20, 30, 40, 50]
cari = 40

low = 0
high = len(data) - 1

while low <= high:
    mid = (low + high) // 2

    if data[mid] == cari:
        print("Data ditemukan")
        break
    elif data[mid] < cari:
        low = mid + 1
    else:
        high = mid - 1
```

4.9 Hubungan Struktur Data dan Efisiensi

Pemilihan struktur data yang tepat akan mempengaruhi kecepatan program, penggunaan memori, dan efisiensi pemrosesan data. Contohnya, pencarian data pada list kecil mungkin tidak terasa lambat, tetapi pada data sangat besar diperlukan algoritma dan struktur data yang lebih efisien.

5. ALAT DAN PERANGKAT

- Laptop / komputer.
- Python menggunakan Google Colab.
- Browser.
- Modul praktikum.

6. PROSEDUR PRAKTIKUM

6.1 Struktur Data Dasar

Langkah Praktikum:

- Buka Google Colab atau IDE Python.
- Buat list sederhana.
- Tambahkan beberapa data.
- Tampilkan isi data.
- Akses elemen tertentu dan amati bagaimana data disimpan.

Contoh Program:

```
data = [10, 20, 30, 40]
print(data)
print(data[0])
```

Tujuan: Mahasiswa memahami konsep penyimpanan data terstruktur.

6.2 Simulasi Operasi Data

Langkah Praktikum:

- Tambahkan data menggunakan `append()`.
- Hapus data menggunakan `remove()`.
- Akses data berdasarkan indeks.
- Tampilkan hasil perubahan.

Contoh Program:

```
data = [10, 20, 30]
data.append(40)
data.remove(20)

print(data)
print(data[0])
```

Tujuan: Mahasiswa memahami manipulasi data pada struktur data linear.

6.3 Abstract Data Type (ADT) Stack

Langkah Praktikum:

- Buat stack sederhana menggunakan list.
- Tambahkan data menggunakan push/append().
- Ambil data menggunakan pop().
- Amati prinsip LIFO.

Contoh Program:

```
stack = []
stack.append("Buku A")
stack.append("Buku B")
print(stack.pop())
```

Tujuan: Mahasiswa memahami konsep ADT Stack.

6.4 Simulasi Queue / Antrian

Langkah Praktikum:

- Buat list sebagai antrian.
- Tambahkan data ke bagian akhir antrian.
- Ambil data dari bagian depan antrian.
- Amati prinsip FIFO.

Contoh Program:

```
queue = []
queue.append("Mahasiswa 1")
queue.append("Mahasiswa 2")
print(queue.pop(0))
```

Tujuan: Mahasiswa memahami konsep ADT Queue.

6.5 Analisis Kompleksitas Linear

Langkah Praktikum:

- Buat list data.
- Gunakan perulangan untuk traversal.
- Hitung jumlah iterasi.
- Bandingkan jumlah langkah saat data bertambah.

Contoh Program:

```
data = [1, 2, 3, 4, 5]
jumlah_iterasi = 0

for i in data:
    jumlah_iterasi += 1
    print(i)

print("Jumlah iterasi:", jumlah_iterasi)
```

Tujuan: Mahasiswa memahami kompleksitas $O(n)$.

6.6 Analisis Kompleksitas Kuadratik

Langkah Praktikum:

- Gunakan nested loop.
- Jalankan program.
- Hitung jumlah proses.
- Bandingkan dengan loop tunggal.

Contoh Program:

```
jumlah = 0
for i in range(5):
    for j in range(5):
        jumlah += 1
        print(i, j)

print("Jumlah proses:", jumlah)
```

Tujuan: Mahasiswa memahami kompleksitas $O(n^2)$.

6.7 Simulasi Linear Search

Langkah Praktikum:

- Buat list angka.
- Tentukan angka yang dicari.
- Cari angka satu per satu.
- Hitung jumlah langkah pencarian.

Contoh Program:

```
data = [5, 10, 15, 20, 25]
cari = 20
langkah = 0

for i in data:
    langkah += 1
    if i == cari:
        print("Data ditemukan")
        break

print("Jumlah langkah:", langkah)
```

Tujuan: Mahasiswa memahami cara kerja pencarian linear.

6.8 Simulasi Binary Search

Langkah Praktikum:

- Gunakan data terurut.
- Tentukan nilai tengah.
- Bandingkan nilai tengah dengan data yang dicari.
- Ulangi pencarian pada bagian kiri atau kanan data.
- Hitung jumlah langkah pencarian.

Contoh Program:

```
data = [10, 20, 30, 40, 50]
cari = 40
low = 0
high = len(data) - 1
langkah = 0

while low <= high:
    langkah += 1
    mid = (low + high) // 2

    if data[mid] == cari:
        print("Data ditemukan")
        break
```

```

elif data[mid] < cari:
    low = mid + 1
else:
    high = mid - 1

print("Jumlah langkah:", langkah)

```

Tujuan: Mahasiswa memahami efisiensi binary search.

6.9 Analisis Efisiensi Algoritma

Langkah Praktikum:

- Bandingkan linear search dan binary search.
- Gunakan jumlah data berbeda.
- Catat jumlah iterasi atau langkah.
- Buat kesimpulan berdasarkan hasil pengamatan.

Contoh Program:

```

# Gunakan hasil langkah pada linear search dan binary search
# Lalu bandingkan jumlah langkah saat jumlah data bertambah.

```

Tujuan: Mahasiswa memahami pentingnya algoritma efisien dalam pengolahan data besar.

7. STUDI KASUS

Studi Kasus 1: Penyimpanan Data Mahasiswa

Sebuah sistem akademik ingin menyimpan daftar nama mahasiswa. Buat program untuk menyimpan data mahasiswa dalam list dan menampilkan seluruh data.

Studi Kasus 2: Pencarian Data Mahasiswa

Mahasiswa diminta mencari data mahasiswa berdasarkan NIM menggunakan linear search.

Studi Kasus 3: Simulasi Stack Buku

Tumpukan buku di perpustakaan mengikuti konsep LIFO. Buat simulasi untuk menambahkan buku dan mengambil buku paling atas.

Studi Kasus 4: Analisis Perulangan

Bandinkan jumlah proses pada single loop dan nested loop, lalu jelaskan perbedaan kompleksitasnya.

Studi Kasus 5: Sistem Perpustakaan

Tentukan struktur data yang cocok untuk daftar buku, riwayat peminjaman, dan pencarian buku.

Studi Kasus 6: Sistem Nilai Mahasiswa

Bandingkan efisiensi pencarian nilai menggunakan metode pencarian yang berbeda.

8. TUGAS PRAKTIKUM

1. Jelaskan pengertian struktur data dan algoritma, serta berikan contoh sederhana penerapannya dalam pemrograman.
2. Buat program menggunakan list untuk menyimpan 5 angka, menampilkan seluruh data, dan melakukan traversal menggunakan perulangan.
3. Buat simulasi ADT Stack dan Queue menggunakan list, dengan operasi dasar seperti menambah data dan menghapus/mengambil data.
4. Buat program pencarian data menggunakan linear search dan binary search, kemudian bandingkan jumlah langkah pencarian dari kedua algoritma tersebut.
5. Buat program nested loop dan analisis kompleksitasnya, kemudian analisis juga kompleksitas dari program berikut:

```
for i in range(10):  
    print(i)
```

Sertakan penjelasan apakah kompleksitasnya termasuk **$O(1)$** , **$O(n)$** , **$O(\log n)$** , atau **$O(n^2)$** .

9. DISKUSI

- Mengapa struktur data penting dalam pemrograman?
- Apa perbedaan struktur data dan ADT?
- Apa perbedaan struktur data linear dan non-linear?
- Mengapa kompleksitas algoritma perlu dianalisis?
- Apa fungsi Big-O Notation?
- Mengapa binary search lebih cepat dari linear search pada data besar dan terurut?
- Mengapa nested loop lebih lambat dibanding single loop?
- Apa dampak algoritma tidak efisien pada data besar?

10. FORMAT LAPORAN PRAKTIKUM

- 1) Judul Praktikum
- 2) Tujuan
- 3) Dasar Teori
- 4) Langkah Praktikum
- 5) Hasil dan Screenshot Program
- 6) Analisis
- 7) Kesimpulan

11. RUBRIK PENILAIAN

Komponen	Bobot
Kehadiran	10%
Kode Program	30%
Hasil	30%
Analisis	30%